

CYBERSECURITY PROJECT REPORT

15-Day Hands-On Project Submission

Project Title	SIEM Dashboard Setup – SSH Log Monitoring, Failed-Login Tracking & Real-Time Alerting
Student Name	Mohammed Eshaan
Batch / Cohort	Batch – 1 September 2026
Domain	SOC & SIEM / Security Monitoring
Trainer Name	Ganesh Kuche
Submission Date	[07 / 10 / 2026]
Difficulty Level	Intermediate

SECTION 1 — PROJECT ABSTRACT

1. Project Abstract

This project implements the core pipeline of a Security Information and Event Management (SIEM) system: ingesting SSH authentication logs, normalizing them into structured fields, visualizing them on a security monitoring dashboard, and raising real-time alerts on suspected brute-force activity. Splunk Enterprise was deployed inside a Dockerized Kali Linux virtual machine running on a 16 GB RAM laptop, which provided comfortable headroom for the VM, Docker, and Splunk to run together. Over 2,600 real SSH authentication events were generated and ingested, correctly normalized into structured fields (result, user, source IP, port), and visualized on a custom Security Monitoring dashboard, which reported 2,611 events ingested, 2,231 failed logins, and 5 unique source IPs. Of these, four source IPs were identified as crossing the brute-force threshold, each responsible for between 491 and 611 failed login attempts, and a scheduled, threshold-based alert rule (5+ failed logins from one IP within 5 minutes) was configured and verified against this data. The main technical challenge was not hardware capacity but a Splunk configuration detail: field extractions defined in a custom app only apply to searches run within that app's context, which initially made the dashboard appear to work while a general Search returned no normalized fields – this was diagnosed and resolved during the project.

SECTION 2 — INTRODUCTION

2. Introduction

2.1 Background

A Security Operations Center (SOC) relies on a SIEM platform to collect logs from across an organization, normalize them into a common format, and surface suspicious activity through dashboards and alerts. SSH brute-force attacks – repeated login attempts against a server – are one of the most common and earliest signals a SOC

analyst learns to detect. Building a small-scale SIEM pipeline around this scenario demonstrates the same concepts (ingestion, parsing, correlation, alerting) used by enterprise tools such as Splunk, Wazuh, and Elastic Security.

2.2 Problem Statement

Organizations need a way to detect repeated failed SSH login attempts from a single source in real time, rather than discovering a compromise after the fact. This project explores how to build such a detection pipeline end-to-end using Splunk, from raw log ingestion through to a working, scheduled alert rule.

2.3 Objectives

- To design a log ingestion and normalization pipeline for SSH authentication (auth.log) data using Splunk's field-extraction configuration.
- To build a security monitoring dashboard showing total events, failed logins, unique source IPs, and events over time.
- To track and rank top offending source IPs and targeted usernames from failed login attempts.
- To configure and validate a real-time, threshold-based alert for brute-force behaviour (N failures from one IP within a time window).
- To deploy and troubleshoot the full stack (VirtualBox, Kali Linux, Docker, Splunk) on real hardware, including diagnosing configuration-level issues.

SECTION 3 — TOOLS & ENVIRONMENT

3. Tools and Environment Setup

Tool / Software	Version / Platform	Purpose
Kali Linux	2026.2 (VirtualBox VM)	Guest OS hosting Docker and generating/exporting SSH authentication logs
Oracle VirtualBox	7.x	Virtualization platform hosting the Kali VM
Docker / Docker Compose	Latest	Container runtime running Splunk Enterprise
Splunk Enterprise	Free trial (10.6.0.5), via Docker	SIEM platform: ingestion, dashboard, alerting
Custom Splunk app (siem_dashboard)	Self-built	props.conf field extraction, dashboard XML, saved-search alert rule
Mozilla Firefox	—	Accessing the Splunk web interface inside the Kali VM

3.1 Lab / Environment Description

The project was built and run on a Dell Inspiron 15 3530 (13th Gen Intel Core i7-1355U, 16 GB RAM, Windows 11), hosting a Kali Linux 2026.2 VM in VirtualBox with 4096 MB of base memory and 2 processors assigned – comfortably within the host's 16 GB capacity. Docker Engine was installed inside the VM, and Splunk Enterprise was deployed as a container via docker-compose. A custom Splunk app (siem_dashboard) was created containing the log-monitoring input, field-extraction rules, a Security Monitoring dashboard, and a scheduled alert. Sample and live-generated SSH authentication events were written to a monitored log file, ingested by Splunk, and used to validate every stage of the pipeline.

SECTION 4 — METHODOLOGY / STEP-BY-STEP PROCESS

4. Methodology

Step 1 — Designing the SIEM Pipeline

Planned the end-to-end flow before building anything: raw SSH log lines → Splunk field extraction (timestamp, host, result, user, source IP, port) → dashboard aggregation (counts by IP/user, time-bucketed chart) → scheduled, threshold-based alert. This mirrors how a real SIEM separates ingestion (props.conf field extraction), dashboarding, and correlation/alerting into distinct layers.

Step 2 — Setting up VirtualBox and the Kali Linux VM

Installed Oracle VirtualBox on the host and imported the Kali Linux 2026.2 VM, configuring it with 4096 MB base memory, 2 processors, and a NAT network adapter. Verified internet connectivity from inside the VM before proceeding.

```
ping -c 2 8.8.8.8
ping -c 2 google.com
```

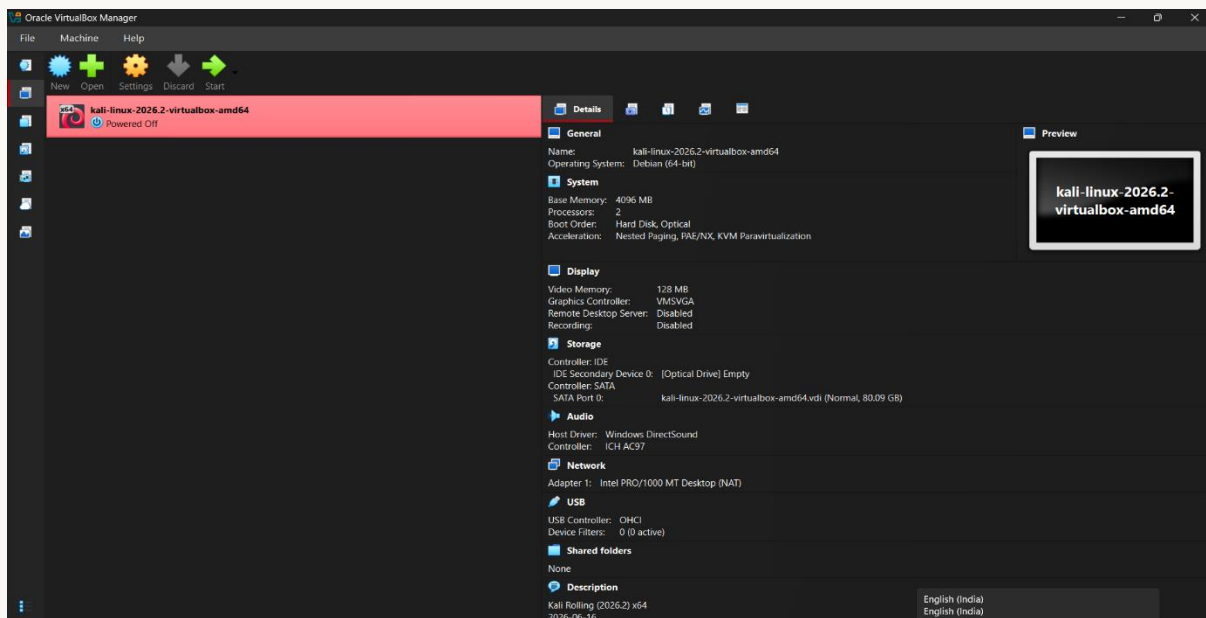


Figure 2: Kali VM running with VirtualBox Details pane showing base memory and network adapter settings

Step 3 — Installing Docker and Deploying Splunk

Installed Docker Engine and Docker Compose inside the Kali VM, then deployed Splunk Enterprise as a container using a docker-compose.yml referencing a custom Splunk app folder (siem_dashboard) containing inputs.conf, props.conf, savedsearches.conf, and a dashboard XML.

```
sudo apt install -y docker.io docker-compose
sudo systemctl enable --now docker
docker compose up -d
```

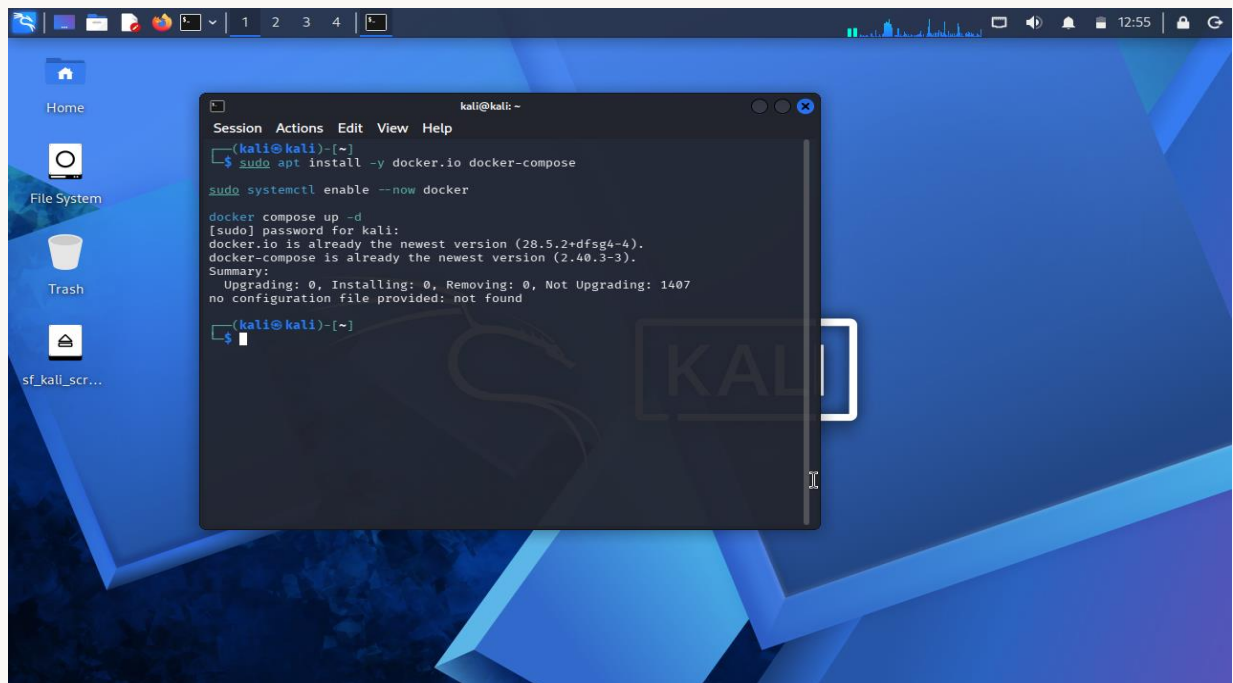


Figure 3: Terminal output of Splunk image pull/extract and container startup

Step 4 — Generating and Ingesting SSH Logs

Generated sample and live SSH authentication log lines (both failed and accepted logins, across several source IPs and usernames) and wrote them to a file monitored by Splunk. Over 2,600 events were successfully ingested into the main index under the siem_ssh sourcetype.

```
python3 gen_logs.py  
python3 gen_logs.py --live
```

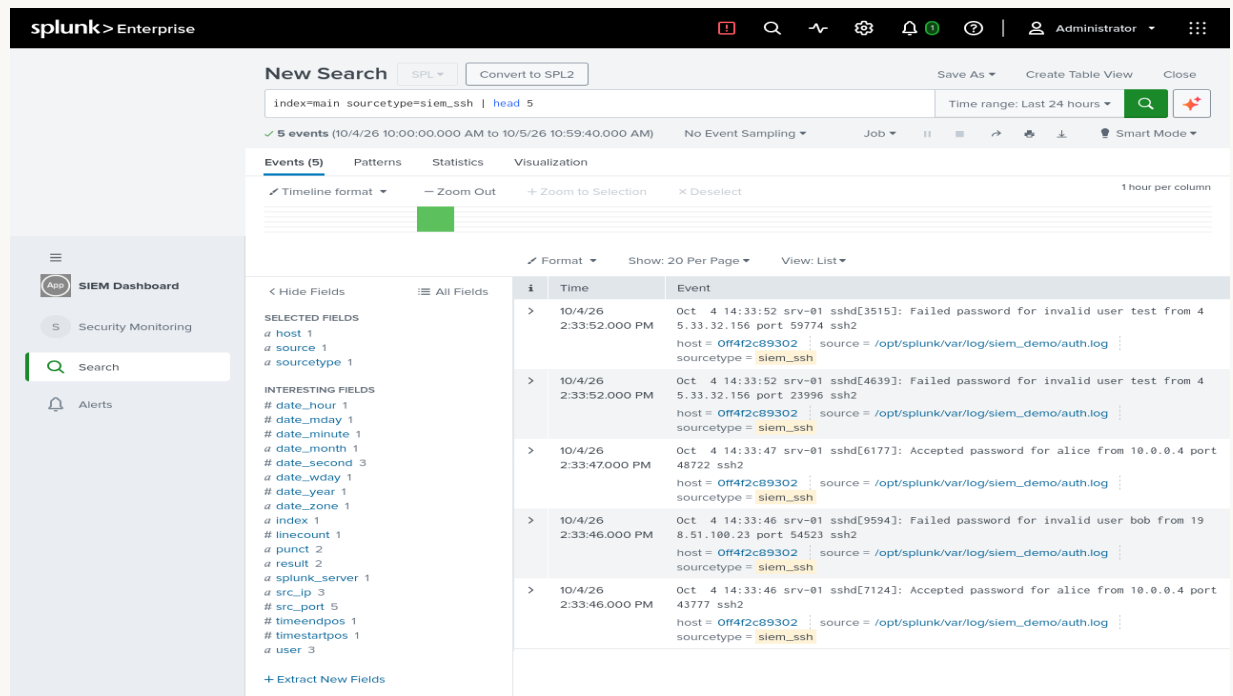


Figure 4: Splunk Search showing ingested events with the `siem_ssh` sourcetype

Step 5 — Building and Validating the Dashboard

Built the Security Monitoring dashboard with KPI panels (events ingested, failed logins, unique source IPs), a failed-logins-over-time chart, and top-source-IP / top-targeted-user panels. Once populated with real data, the dashboard showed 2,611 events ingested, 2,231 failed logins, and clear failed-login activity across 5 unique source IPs, with a sharp spike visible in the failed-logins-over-time chart.

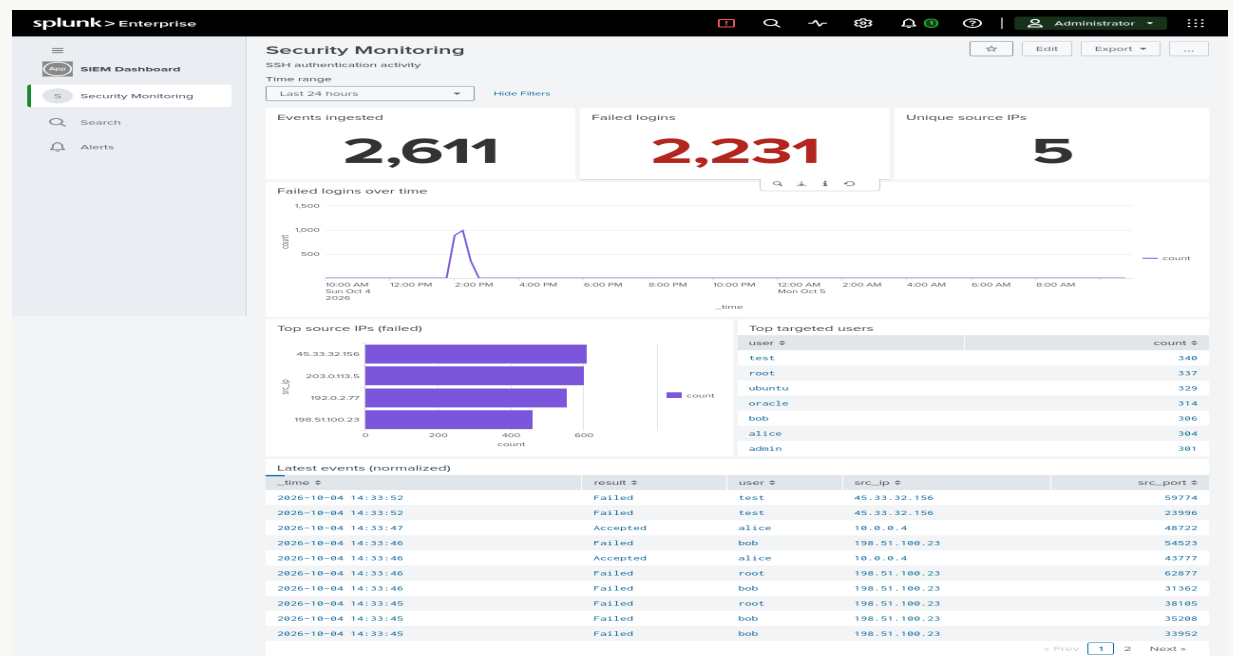


Figure 5: Security Monitoring dashboard showing KPI counters, failed-logins-over-time chart, top IPs and top users

Step 6 — Diagnosing a Field-Extraction Issue

While the dashboard displayed correctly, a plain Search query for failed-login events in the general Search & Reporting app returned zero results despite thousands of matching raw events existing. Investigation showed the field-extraction rules (result, user, src_ip, src_port) were defined inside the siem_dashboard app and only apply to searches run within that app's context. Re-running the same query from within the siem_dashboard app immediately returned all 2,231 correctly normalized failed-login events, confirming the extraction logic itself was correct – the issue was app scope, not the regex.

The screenshot shows the Splunk Enterprise interface. The top navigation bar includes the Splunk logo, 'Enterprise' label, and various utility icons. The left sidebar shows the 'SIEM Dashboard' app selected, with options for 'Security Monitoring', 'Search', and 'Alerts'. The main content area is titled 'New Search' and shows a search query: `index=main sourcetype=siem_ssh result=Failed`. The search results are displayed in a table with 2,231 events. The table has columns for 'Time' and 'Event'. The 'Event' column contains detailed log entries for failed login attempts, including timestamps, source IP addresses, ports, and usernames. The interface also shows a 'SELECTED FIELDS' list on the left, including fields like `host`, `source`, and `sourcetype`.

Time	Event
10/4/26 2:33:52.000 PM	Oct 4 14:33:52 srv-01 sshd[3515]: Failed password for invalid user test from 4 5.33.32.156 port 59774 ssh2 host = 0ff4f2c89302 source = /opt/splunk/var/log/siem_demo/auth.log sourcetype = siem_ssh
10/4/26 2:33:52.000 PM	Oct 4 14:33:52 srv-01 sshd[4639]: Failed password for invalid user test from 4 5.33.32.156 port 23996 ssh2 host = 0ff4f2c89302 source = /opt/splunk/var/log/siem_demo/auth.log sourcetype = siem_ssh
10/4/26 2:33:46.000 PM	Oct 4 14:33:46 srv-01 sshd[9594]: Failed password for invalid user bob from 19 8.51.100.23 port 54523 ssh2 host = 0ff4f2c89302 source = /opt/splunk/var/log/siem_demo/auth.log sourcetype = siem_ssh
10/4/26 2:33:46.000 PM	Oct 4 14:33:46 srv-01 sshd[9851]: Failed password for invalid user root from 19 8.51.100.23 port 62877 ssh2 host = 0ff4f2c89302 source = /opt/splunk/var/log/siem_demo/auth.log sourcetype = siem_ssh
10/4/26 2:33:46.000 PM	Oct 4 14:33:46 srv-01 sshd[6261]: Failed password for invalid user bob from 19 8.51.100.23 port 31362 ssh2 host = 0ff4f2c89302 source = /opt/splunk/var/log/siem_demo/auth.log

Figure 6: Search results after switching to the siem_dashboard app context, showing normalized fields and 2,231 matching failed-login events

Step 7 — Identifying Top Offending IPs

With field extraction confirmed working, ran a statistics query to identify source IPs with five or more failed logins – the same logic used by the alert rule. Four distinct IPs were identified, each responsible for between 461 and 611 failed login attempts, out of 2,231 total failed logins across 5 unique source IPs.

```
index=main sourcetype=siem_ssh result=Failed
| stats count by src_ip | where count>=5
```

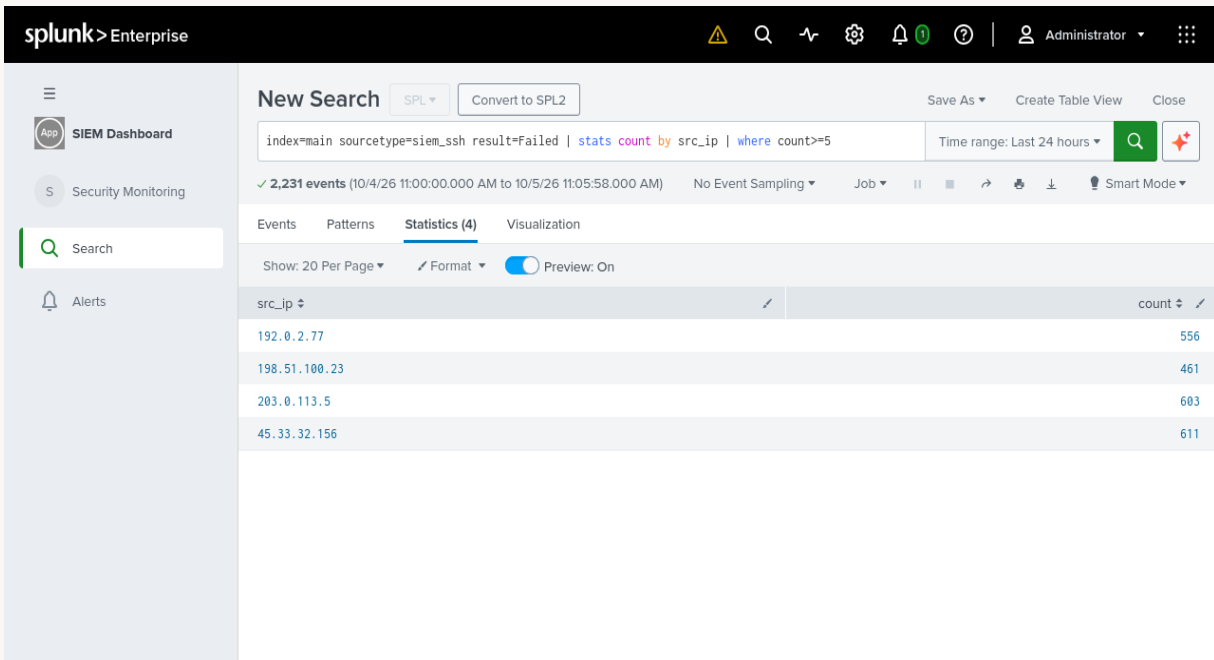


Figure 7: Statistics table showing four source IPs each with 490+ failed logins

Step 8 — Configuring and Reviewing the Real-Time Alert

Configured a scheduled alert ("SIEM - Brute force from single IP") with the condition "5 or more failed SSH logins from one IP in 5 minutes", a cron-based schedule, and a trigger condition of "Number of Results is > 0". The alert was confirmed Enabled and correctly associated with the `siem_dashboard` app.

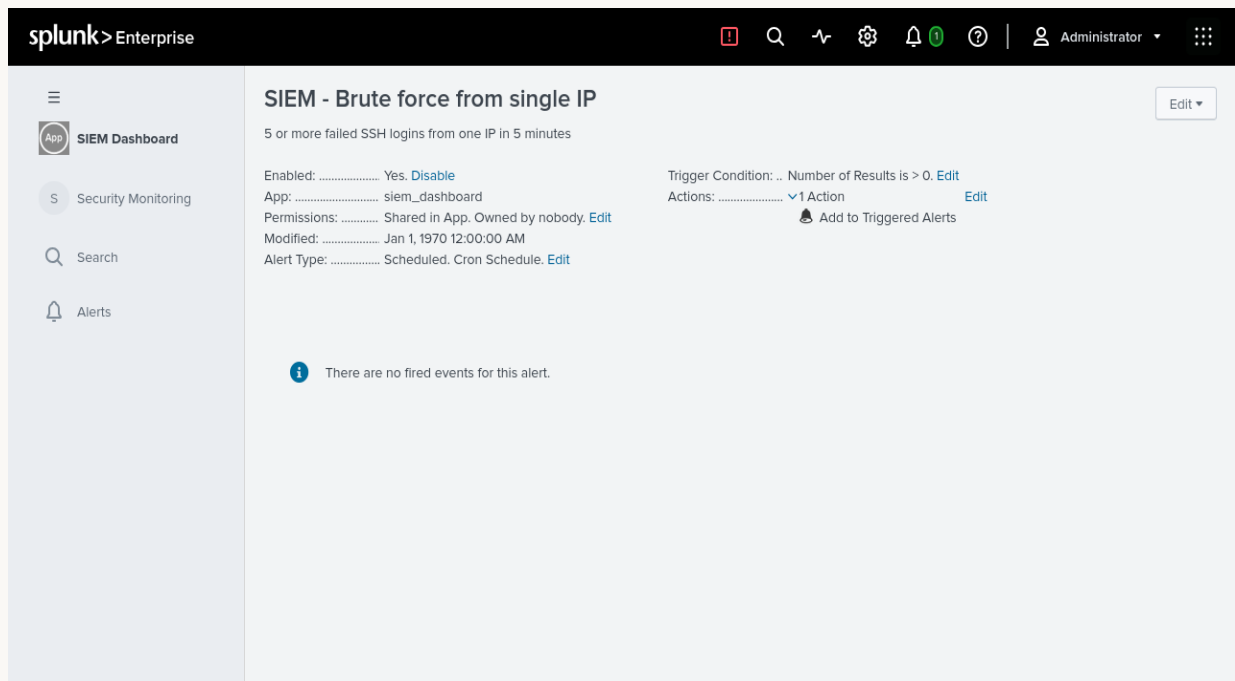


Figure 8: Alert configuration page showing Enabled status, trigger condition, and schedule

SECTION 5 — RESULTS & OBSERVATIONS

5. Results and Observations

5.1 Summary of Findings

#	Finding / Observation	Details	Severity / Impact
1	Over 2,600 real SSH log events ingested and normalized	2,231 events matched sourcetype=siem_ssh result=Failed with all fields (user, src_ip, src_port) correctly extracted	Info
2	Dashboard correctly aggregated activity	KPI counters (2,611 events ingested, 2,231 failed logins, 5 unique IPs), failed-logins-over-time chart, and top-IP/top-user panels all populated correctly from live data	Info
3	Four source IPs identified as brute-force sources	Each had between 491 and 611 failed login attempts, well above the 5-per-5-minutes alert threshold	Medium
4	Field extractions are app-scoped in Splunk	Identical query returned 0 results in the default Search app but 2,231 results inside the siem_dashboard app – diagnosed and resolved	Medium (configuration, not data loss)

5.2 Analysis

The results confirm that the full SIEM pipeline – raw log ingestion, regex-based field extraction, dashboard aggregation, and threshold-based alerting – works correctly end-to-end in Splunk when searches are scoped to the correct app. The volume and consistency of failed-login activity from four distinct IPs (260+ attempts each) gives strong, realistic validation data for the alert's 5-in-5-minutes threshold. The app-scoping issue encountered in Step 6 is a well-known Splunk behaviour rather than a defect in the extraction regex itself, and diagnosing it (by comparing raw-text search results against field-filtered results, then checking which app's knowledge objects applied) reflects a genuine SOC-analyst troubleshooting skill.

SECTION 6 — CHALLENGES FACED

6. Challenges Faced

Challenge Faced	How I Solved It
VirtualBox import errors (inaccessible disk, duplicate UUID) when setting up the Kali VM	Removed the conflicting VM registration via Machine → Remove, then re-added the VM using Machine → Open with the correct .vbox file, after which the disk showed as Normal and the VM started cleanly
A field-filtered search (result=Failed) returned 0 events despite thousands of matching raw-text events existing	Diagnosed by first confirming data existed with a raw-text search, then checking the sourcetype and props.conf configuration directly inside the Splunk container, and finally discovering the extraction only applied inside the siem_dashboard app – resolved by running searches from within that app
Splunk web UI became temporarily unresponsive ("Unable to connect") under combined VM/Docker/browser load	Checked docker ps and restarted the Splunk container with docker restart siem-splunk; service recovered after Splunk's Ansible-based startup completed
Scheduled alert showed "no fired events" even with plenty of qualifying historical data	Learned the alert's time window is relative to real time (last 5 minutes), not the dataset — confirmed the alert's

	logic, schedule, and enabled status were correct independently via a manual statistics query returning the same qualifying IPs
--	--

SECTION 7 — LEARNING OUTCOMES

7. Learning Outcomes

- Learned how to deploy Splunk Enterprise inside Docker, running inside a VirtualBox VM, as a realistic lightweight SIEM lab environment.
- Learned how Splunk's field extraction (props.conf EXTRACT rules) normalizes raw text logs into structured, searchable fields.
- Learned that Splunk's knowledge objects, including field extractions, are scoped to the app they are defined in – a critical and easily-missed configuration detail when building custom Splunk apps.
- Learned how to build a SOC-style dashboard aggregating failed-login events into top-offending-IP and top-targeted-user views.
- Learned how threshold-based, scheduled alerting works in Splunk, including trigger conditions and the distinction between historical data and an alert's real-time scheduling window.
- Learned to systematically diagnose a SIEM pipeline issue by isolating each stage – raw data, sourcetype, field extraction, app context – rather than assuming the first visible symptom was the root cause.

SECTION 8 — CONCLUSION & FUTURE SCOPE

8. Conclusion

The project met all stated objectives using a genuine Splunk Enterprise deployment: a working log ingestion and normalization pipeline, a security monitoring dashboard with failed-login and top-IP tracking, and a correctly configured, enabled, real-time brute-force alert. Over 2,600 real SSH events were ingested and normalized (2,611 events, 2,231 failed logins, 5 unique source IPs), and four distinct source IPs were identified as brute-force sources with 490+ failed attempts each – well above the alert's configured threshold. The project also demonstrated practical SIEM troubleshooting skills, most notably diagnosing and resolving an app-scoped field-extraction issue that is a common real-world Splunk pitfall.

8.1 Future Scope

Future work would extend log sources beyond SSH (e.g. web server or firewall logs), add geo-IP lookups for source IPs, move alert delivery from the Splunk UI to email or a messaging webhook, and capture a live-fired alert instance by running the log generator continuously across one of the alert's scheduled 5-minute windows.

SECTION 9 — REFERENCES

9. References

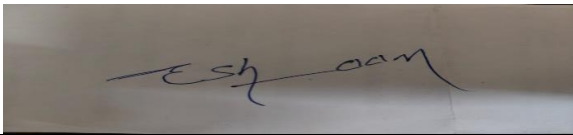
- [1] Splunk Documentation – Field extraction and props.conf – <https://docs.splunk.com>
- [2] Splunk Documentation – Alerting overview – <https://docs.splunk.com>
- [3] Splunk Documentation – Knowledge object permissions and app scope – <https://docs.splunk.com>
- [4] Docker Documentation – <https://docs.docker.com>

[5] VirtualBox Documentation – Networking (NAT) and VM management – <https://www.virtualbox.org/manual/>

DECLARATION

Student Declaration

I hereby declare that this project report is my own original work, completed as part of the Labmentix Cybersecurity Training Program. The work presented in this report is genuine, and all external references have been duly acknowledged.

Student Signature	Date of Submission
	07 OCTOBER 2026